

IFQ580: Machine Learning

Assessment 1A

Learning Group # 02H

STUDENT NAME	STUDENT ID
Nev Doyle	n12717991
Alen James	n12837075
Ananth Naidu	n12668605

Contribution Matrix

	Nev Doyle	Alen James	Ananth Naidu
Nev Doyle	100%	100%	100%
Alen James	100%	100%	100%
Ananth Naidu	100%	100%	100%

Task 1 — Exploratory Data Analysis and Preprocessing

1.1 Dataset overview and exploratory analysis

The purpose of this exploratory data analysis (EDA) is to understand the characteristics of the kick_1 dataset, identify quality issues, and investigate relationships between variables. This understanding will also contribute to identifying preprocessing requirements before machine learning modelling begins.

Table 1-1: Initial data type query of kick_1.csv data frame

DATASET NAME	KICK_1.CSV
SOURCE	List of Variables (2026), courtesy of Professor Richi Nayak, School of Computer Science, QUT
NUMBER OF OBSERVATIONS	41,476
NUMBER OF VARIABLES	31
TARGET VARIABLE	IsBadBuy (1 = Bad Buy/Kick, 0 = Not Bad Buy)

Dataset Overview

Initial inspection showed a mixture of numerical and categorical variables. Several MMR price variables were imported as text/object values rather than numeric values, indicating that invalid or non-numeric entries required investigation before modelling.

Table 1-2: Summary of data types contained in kick_1.csv

DATA TYPE	RESULT
DISCRETE (INTEGER)	3
CONTINUOUS (FLOAT)	3
STRING	24
OBJECT (GROUP)	1

Data Exploration

Our team explored the dataset to understand its structure, the variables, data types, distribution and overall quality prior to commencing preprocessing. This activity involved:

- Identifying missing or unusual variables
- Reviewing variable distributions and unique values
- Discovering and reviewing potential outliers
- Comparing related or similar variables for possible redundancy exclusion
- Examining the distribution of the target variable IsBadBuy

Key Data-Quality Findings

Table 1-3: Summary key findings contained in kick_1.csv during EDA

VARIABLE / ISSUE	KEY FINDING	IMPLICATION
Missing observations	A recurring group of 44 observations consisted of missing values across many unrelated variables. This may suggest record corruption	These observations represented a fractional population of the overall variables size and contained little predictor information.

		Outcome: These were later removed during pre-processing.
PRIMEUNIT / AUCGUART	Approximately 96% of observations contained unknown (?) values. Imputation was not considered a viable option as the model would have significantly depended on data had been estimated rather than observed.	As approximately 96% of the values were unknown, too little valid information remained for these variables could be used to reliably contribute to predicting IsBadBuy Outcome: These were later removed during pre-processing.
WheelTypeID / WheelType	Cross-tabulation showed that the two variables represented substantially the same wheel-type information shows that WheelTypeID is very likely the numeric code for WheelType. They may be two representations of the same feature, which is supported by in the assignment. One of these variables may potentially be a redundant variable. Without assuming, cross tab validation was used to compare two categorical variables and count how often each combination occurs. This shows that 20406 cars with WheelTypeID = 1 have WheelType = Alloy, similar mapping exists for Covers and Special. There a very clear mapping can be observed.	This finding indicated redundancy between the variables as they contained the same information in different representations. Cross Tab demonstrates that WheelTypeID and WheelType are essentially duplicate representations of the same information. Outcome: Remove WheelTypeID as it contained numeric identifiers.
Transmission	The categorical values MANUAL and Manual represented the same transmission type but used inconsistent capitalisation	The values were standardised to uppercase so that MANUAL and Manual were treated as a single MANUAL category rather than two separate categories. Outcome: standardise Transmission variable during pre-processing.
Each observation had a unique PurchaseID	Each observation had a unique PurchaseID	This confirms that this variable is just an identifier and every unique observation does not help in identifying if a vehicle is a bad buy risk. Outcome: Exclude PurchaseID during pre-processing
ForSale	The variable was effectively constant after accounting for inconsistent capitalisation of Yes.	The sum of all Yes, YES and yes observations is 41470 which represents 99.99% of the dataset. As the predictor needs variation to classify an outcome as a "Bad Buy", it cannot split or classify the data. We can exclude ForSale due to non-near zero variance. Outcome: Exclude ForSale during pre-processing
PurchaseTimestamp / PurchaseDate	Both represented the same purchase-time information in different formats. Comparison of variables for the first 20 outputs without duplicates yields: Variables PurchaseTimeStamp and PurchaseDate may have potential redundancies this is evidenced by observing the first rows which shows PurchaseTimeStamp PurchaseDate 1253232000 18/09/2009 10:00.	This output shows that the purchase time information is represented in two formats. PurchaseDate is easier to interpret and will be highlighted to keep. Outcome: Exclude PurchaseTimeStamp and retain PurchaseDate as it is more interpretable.

MMR valuation variables	Several fields contained systematic 0/1 or non-numeric values inconsistent with genuine vehicle prices. The MMR variables were converted to numeric values. The non-numeric NaN values were converted to missing values. Invalid numeric values such as ? Were replaced with NaN. This allowed the missing values to be imputed. The list of MMR affected are: • MMRAcquisitionAuctionAveragePrice • MMRAcquisitionAuctionCleanPrice • MMRAcquisitionRetailAveragePrice • MMRAcquisitionRetailCleanPrice (note the dataset's spelling of "Acquisition") • MMRCurrentAuctionAveragePrice • MMRCurrentAuctionCleanPrice • MMRCurrentRetailAveragePrice • MMRCurrentRetailCleanPrice • MMRCurrentRetailRatio The invalid and missing values in each of these MMR variables were imputed using the median characteristic. The median was selected because MMR variables extreme outlier vehicle price values that could skew mean to the right if chosen. As the median is less sensitive to outliers, it is a more appropriate choice.	MMR Variables required cleaning and imputation before modelling. Outcome: Imputed total number of missing MMR observations by observing the statistical distribution
Color/Nationality/Size	Most observations contained valid colour categories; however, NOT AVAIL and ? both represented unavailable colour information.	These were standardised into a single UNKNOWN category so the observations could be retained. The ? values were changed to UNKNOWN. Other categories were retained because there was insufficient evidence to combine them. Outcome: standardise these variables during pre-processing
VehBCost	29 observations contained invalid '?' values. 29 represents a fractional proportion of observations so the data can be imputed statistically.	Outcome: VehBCost was converted to numeric and the 29 missing values (~0.07%) were imputed using the median of \$6,710, which was less sensitive to extreme values.
IsOnlineSale	This variable is data type "object" which pandas flagged earlier as containing multiple mixed types. The IsOnlineSale variable was expected to be binary, where 0 represents a non-online sale and 1 represents an online sale. However, value_counts() identified several unexpected values, including -1.0, 2.0, 4.0, ?, and missing values. The values 0 and 0.0 appear to be the same category, as do 1 and 1.0. The remaining values fall outside the documented binary categories and were therefore treated as invalid rather than assuming their intended meaning. The meaning of -1.0 could not be established from the available documentation, so it was not recoded as 1 because this would introduce an unsupported assumption. It was treated as a invalid value that would be imputed in the future. Instead, invalid values were treated as missing and subsequently imputed using the mode (0), which was the dominant valid category.	Values were observed outside the binary categories, which suggests a data quality issue. Categories outside this range were classified invalid as they could not be reliably categorised. The variation in 0 and 1 showed sufficient variation for IsOnlineSale to be retained as a useful predictor. Outcome: Convert invalid values -1.0, 2.0, 4.0 and ? Into NaN. Then impute NaN to enable the predictor to be retained for modelling.

IsBadBuy	Approximately 87.05% of observations were Not Bad Buy and 12.95% were Bad Buy	The target variable is imbalanced and this needs to be considered when evaluating model performance. Therefore, when splitting the dataset into test and training data sample populations, both data sets need to be stratified as preserve the original populations.
----------	---	---

1.2 Variable selection and preprocessing

Following EDA, the initial set of 31 variables were either excluded or retained depending on their role, quality of data and value associated with being a predictor for IsBadBuy.

Retained Variables

Categorical variables that described characteristics of the vehicle, auction and purchase process that could potentially contribute to predicting whether a vehicle was a 'bad buy'. Although IsOnlineSale and IsBadBuy were numerically coded as 0 and 1, they were treated as nominal binary variables because the values represented categories rather than numerical quantities.

Table 1-4: Retained Categorical Variables

VARIABLE	ROLE	MEASUREMENT LEVEL	JUSTIFICATION
Auction	Input	Nominal	Identifies the auction provider and may capture differences between auction sources.
Make	Input	Nominal	Identifies the vehicle manufacturer and may be associated with the likelihood of a bad buy.
Color	Input	Nominal	Represents vehicle colour as unordered categories.
Transmission	Input	Nominal	Distinguishes transmission types such as automatic and manual.
WheelType	Input	Nominal	Identifies the vehicle's wheel type as unordered categories.
Nationality	Input	Nominal	Groups vehicles according to manufacturer nationality/category.
Size	Input	Nominal	Represents vehicle size categories such as compact, medium and large.
VNST	Input	Nominal	Identifies the US state associated with the vehicle purchase.
IsOnlineSale	Input	Nominal (binary)	Indicates whether the vehicle was purchased online (1) or not (0).
IsBadBuy	Target	Nominal (binary)	Indicates whether the purchased vehicle was a Kick/bad buy (1) or not (0).

Numerical variables describing vehicle age, usage, acquisition cost, warranty cost and market valuations because these characteristics could reasonably contribute to identifying potential bad buys were retained.

Table 1-5: Retained Numerical Variables

VARIABLE	ROLE	MEASUREMENT LEVEL	JUSTIFICATION
PurchaseDate	Input	Interval	Contained variation which may capture time-related changes in market and auction conditions.
VehYear	Input	Interval	Contained variation which may capture age-related differences associated with Bad Buy risk.
VehOdo	Input	Ratio	Contained variation which may reflect vehicle usage, condition and Bad Buy risk.

MMRAcquisitionAuctionAveragePrice	Input	Ratio	Sufficient variation which may help identify Bad Buy risk.
MMRAcquisitionAuctionCleanPrice	Input	Ratio	Sufficient variation which may help identify Bad Buy risk.
MMRAcquisitionRetailAveragePrice	Input	Ratio	Sufficient variation which may help identify Bad Buy risk.
MMRAcquisitionRetailCleanPrice	Input	Ratio	Sufficient variation which may help identify Bad Buy risk.
MMRCurrentAuctionAveragePrice	Input	Ratio	Sufficient variation which may help identify Bad Buy risk.
MMRCurrentAuctionCleanPrice	Input	Ratio	Sufficient variation which may help identify Bad Buy risk.
MMRCurrentRetailAveragePrice	Input	Ratio	Sufficient variation which may help identify Bad Buy risk.
MMRCurrentRetailCleanPrice	Input	Ratio	Sufficient variation which may help identify Bad Buy risk.
MMRCurrentRetailRatio	Input	Ratio	Sufficient variation which may help identify Bad Buy risk.
VehBCost	Input	Ratio	Contained variation in acquisition cost that may help identify financially unfavourable vehicle purchases.
WarrantyCost	Input	Ratio	Contained variation in warranty cost that may reflect differences in expected vehicle risk.

Excluded Variables

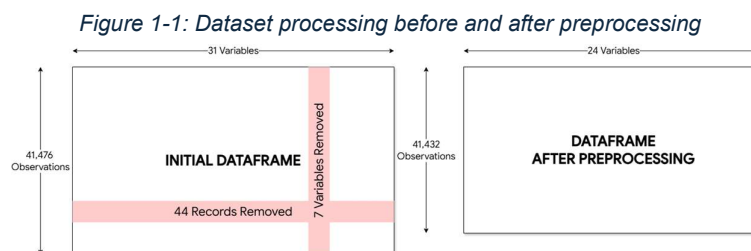
Variables that were classified as identifiers or redundant with retained variables, contained insufficient usable information, or showed near zero variation were excluded.

Table 1-6: Retained Numerical Variables

VARIABLE	ROLE	JUSTIFICATION
PurchaseID	Excluded	Unique identifier that does not describe a vehicle characteristic and was not considered meaningful for prediction
PurchaseTimestamp	Excluded	Contained redundant purchase-time information already represented by the more interpretable PurchaseDate
WheelTypeID	Excluded	Coded representation of wheelType; cross-tabulation showed the variables contained redundant information
TopThreeAmericanName	Excluded	Broad manufacturer grouping that overlapped with the more detailed information retained in Make
PRIMEUNIT	Excluded	Approximately 95.6% of values were unknown (?), leaving insufficient usable information
AUCGUART	Excluded	Approximately 95.6% of values were unknown (?), leaving insufficient usable information
ForSale	Excluded	Almost all observations represented Yes, resulting in near-zero variation and little potential predictive information

1.3 Dataset Before and After Preprocessing

The figure below illustrates the dimensional transformation of the kick_1.csv data frame after preprocessing.



1.4 Target Distribution After Preprocessing

The target variable's (IsBadBuy) binary nominal distribution was compared before and after pre-processing. Bad buys represented approximately 12.95% of observations both before and after preprocessing, indicating that removal of the 44 highly incomplete observations did not materially alter the target distribution. The target was substantially imbalanced, with approximately 87% non-bad buys and 13% bad buys, which should be considered when evaluating subsequent classification models.

Table 1-7: Target variable before and after preprocessing distribution

ISBADBUY	BEFORE PREPROCESSING	AFTER PREPROCESSING
0 — Not bad buy	36,105 (87.05%)	36,067 (87.05%)
1 — Bad buy/Kick	5,371 (12.95%)	5,365 (12.95%)
Total	41,476	41,432

Task 2 — Decision Tree Modelling

2.0 Model preparation

To create a modelling dataframe, PurchaseDate was removed as it was not possible to use datetime values (contained within PurchaseDate) in the decision tree. Additionally, the practical modelling process requires numerical inputs, so there was no way to represent date information in the decision tree model.

One-Hot-Encoding (OHE) was then used to transform the categorical predictor variables into binary numerical features so that it could be used in the decision tree model. As a result, it increased the number of columns to 121.

The modelling dataset was then separated into the target (y) and predictor features (X). IsBadBuy was used target since the goal of the model is to classify vehicles as either a Bad Buy or a non-Bad Buy. Feature names were retained for future use of the decision tree and the predictor data was converted to a numpy array.

The decision was made to split the data into 70% training and 30% testing data subsets by setting a value of 0.3 for `test_set_size`. This was considered a suitable initial split metric as it allowed most of the data to be used for model development and still have enough unseen data to evaluate the models' effectiveness. Since `random_state` was defined as 10, it would ensure that the same randomisation sequence was followed thus making it reproducible.

Given that the target variable was unbalanced based on the previous task's preprocessing results, stratification was required performed to maintain approximately the same proportion of `IsBadBuy = 1` and `IsBadBuy = 0` observations within the training and test datasets. Following data partitioning, it was determined whether each observation within X had a corresponding observation within y and if the data remained properly aligned.

Following the correct 70/30 split between the training and test sets, it was confirmed that there were 29,002 observations within the training set and 12,430 observations within the test set.

Figure 2-1: X and y training and test set sizes

```
Size of X_train: 29002
Size of y_train: 29002
Size of X_test: 12430
Size of y_test: 12430
```

To confirm that class distributions were maintained following stratification, normalised count values were compared for the target variable (IsBadBuy) in both the training and test datasets.

Figure 2-2: Preservation of test and training proportions of y via stratification

```
Training proportions:
IsBadBuy
0    0.870526
1    0.129474
Name: proportion, dtype: float64

Test proportions:
IsBadBuy
0    0.870475
1    0.129525
Name: proportion, dtype: float64
```

The above output shows that both proportions were consistent with those found within the original target distribution. Thus, stratification successfully preserved class distributions throughout both datasets.

Task 2.1(A) - Parameters used for Default Decision Tree (DDT)

A decision tree classifier was created using default hyperparameters as a base model to establish a starting point for further comparison. The model was developed using the training dataset (x_train and y_train). Random_state = 10 was again used to enable reproducibility and consistency with prior practical exercises.

Decision trees have many model parameters that can affect their size and construction. The default model parameters obtained from the code output are summarised in the table below.

Table 2-3: Summary of parameters used for the Default Decision Tree

PARAMETER	VALUE	DESCRIPTION
criterion	gini	uses Gini impurity value to determine quality of split
splitter	best	Determines best available split at each node
max_depth	None	No maximum depth indicates that the tree is allowed to grow until other stopping conditions are reached
min_samples_split	2	At least two observations to split internal nodes
min_samples_leaf	1	Leaf node can contain minimum of one observation
max_features	None	Can consider all possible features when searching for best split
max_leaf_nodes	None	No maximum number of terminal leaves specified
min_impurity_decrease	0.0	Must find improving split beyond meeting valid reduction threshold in impurity
min_weight_fraction_leaf	0.0	Must have minimum weighted fraction of samples within leaves
class_weight	None	Does not apply additional weighting to either target classes
ccp_alpha	0.0	Does not perform cost complexity pruning
monotonic_cst	None	Does not constrain monotonic relationships between any variables
random_state	10	Seed used to make model reproducible

Task 2.1(B): Data split used to create training and test sets

The dataset was divided into 70 % training data and 30 % test data to provide sufficient observations for model development while providing a substantial independent dataset for evaluating model performance. The relatively large test sample set (12,400) allowed for model performance to be assessed with greater confidence. The dataset was imbalanced with approximately 13 % of observations representing bad buys in the `IsBadBuy` variable. A substantial independent test set allows more bad buys to be identified with the model. A split of 80 % training and 20 % test sets (8,300) would have reduced the test number of bad buys by approximately 500 observations reducing confidence levels.

Task 2.1(C): classification accuracy on the training and test datasets

Classification accuracy of the default decision tree was measured using both the training and test datasets. Comparison of classification accuracy on both datasets is useful for assessing whether model learned patterns within training data effectively and whether model performs similarly well on unseen test data. Accuracy was calculated by calling `score()` on the training and test datasets separately. Both metrics provide insight into potential model overfitting issues.

Figure 2-4: Training and test accuracies for Default Decision Tree

```
Training accuracy: 1.0
Test accuracy: 0.8178600160901046
```

The default decision tree reported a training accuracy of 100%, but a test accuracy of 81.79%. As noted earlier, perfect training accuracy suggests that model correctly classified every single record within the training dataset. Approximately an 18.21% decrease occurred however when applying the model to unseen test records. The significant drop off between training accuracy and test accuracy strongly suggests that the DDT is fitting too close to the training data and therefore is likely overfitting.

Overfitting is a common result of models with low bias and high variance. Models with low bias fit very well to training data due to extreme closeness in fit, therefore produce minimal errors in prediction when predicting against training data (low train error). However, models with high variance tend to vary greatly with even slight variations in input data and therefore predict poorly when tested against new/unseen data.

Task 2.1(D): Size of the DDT

The size and complexity of the DDT was analysed by identifying the total number of nodes, number of leaf nodes and number of decision rules. Number of leaf nodes were retrieved using `get_n_leaves()`.

Figure 2-5: Size of Default Decision Tree

```
Number of nodes: 6303
Number of leaves: 3152
```

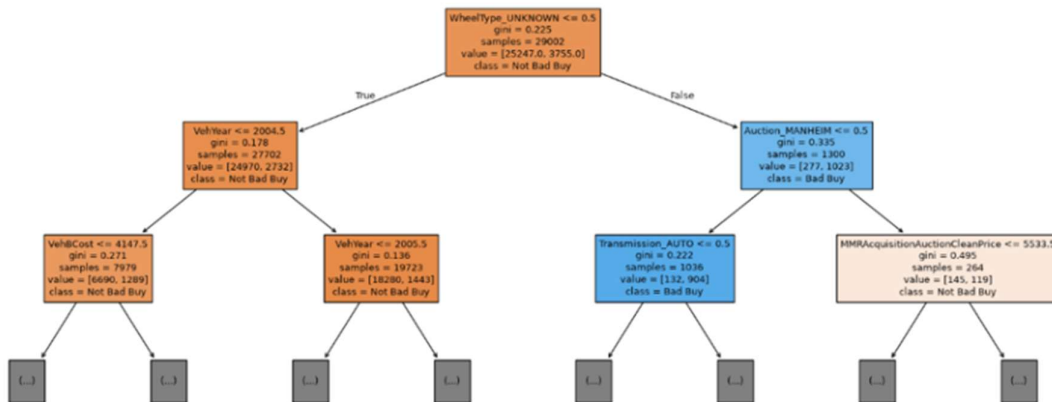
There were 6,303 total nodes in the DDT with 3,152 being leaf nodes. Every leaf corresponds to one classification rule that defines an end-of-path from root node down to leaf node for a particular vehicle. Therefore, every root-to-leaf path may define one decision rule and conversely every decision rule ends in precisely one leaf node.

As a result, each leaf node defines one decision rule and vice versa, establishing a one-to-one correspondence between paths down the tree. This demonstrated that the DDT is complex as evidenced by its large number of nodes and decision rules and is consistent with it having been allowed to grow without limits as indicated by its unlimited depth hyperparameter as well as evidence of overfitting presented by large difference between training and test accuracies.

Task 2.1(E): Variables used for first and second split

The upper parts of the decision tree were analysed where first and second splits occur along with each of the two branches from first splits. With 6,303 nodes present in entire tree, a decision was made to only plot the top-most elements of tree sufficient for interpretation by using `plot_tree()` with `max_depth=2`. This produces the following decision tree shown in the figure below.

Figure 2-6: View of DDT at `max_depth = 2`



Each node in the DDT displayed information such as splitting condition, Gini impurity measure, sample count (n), distribution of target class values (value), and majority class label (class). Less intense colours indicate less homogeneous mixtures of classes; stronger colours indicate more homogeneous mixtures of classes. Orange colour indicates classification as closer to `IsBadBuy = 0`. The Blue colour indicates classification closer to `IsBadBuy = 1`.

The DDT generated by the algorithm chose `WheelType_UNKNOWN` as the first split variable using a threshold of 0.5. There were no specific manual decisions made regarding selection of variables/thresholds for splitting. During training, the algorithms searched among all candidate features to find split(s) that resulted in largest reduction in Gini impurities (i.e., largest increase in homogeneity).

`WheelType_UNKNOWN` was the variable used for the first level split and is a binary OHE feature. A threshold of 0.5 identifies vehicles that have wheel types documented (`WheelType_UNKNOWN = 0`) vs. Vehicles whose wheel types are unknown (`WheelType_UNKNOWN = 1`). Vehicles that meet first condition (wheel type is documented) take left branch after splitting; vehicles that don't meet first condition (wheel type is unknown) take right branch after splitting.

After the first split, the algorithm created second level splits on each branch from the first split. For vehicles whose wheel types are known, the algorithms chose `VehYear` as second level split variable with a threshold of 2004.5. For vehicles whose wheel types are unknown,

the algorithm chose Auction_MANHEIM as second level split variable with a threshold of 0.5. Therefore, like first level splits, second level splits were chosen automatically by algorithms during training instead of being manually selected.

Task 2.1(F): Top five important DDT variables

To determine which five attributes were most useful for creating the default decision tree, a custom `display_feature_importances()` function was developed, based on the same procedure outlined in the practical exercises. The `display_feature_importances()` function retrieved the feature importances computed through the fitted decision tree via `feature_importances_` attribute in `sklearn`. Using `numpy`'s `argsort()` and `flip()`, the attributes were ordered in descending order of their importance. Each feature importance measures the degree to which each attribute contributes to the Gini impurity reduction over all nodes of the tree.

Figure 2-7: Top five important variables for DDT

```
WheelType_UNKNOWN : 0.17996699608597516
VehOdo : 0.07061515936045826
VehBCost : 0.06784471447716978
MMRCurrentRetailRatio : 0.055102054409467804
MMRAcquisitionAuctionAveragePrice : 0.04733111995764246
```

WheelType_UNKNOWN was identified as the attribute with greatest influence in developing the default decision tree with an importance measure of approximately .180. Therefore, it is determined that approximately eighteen percent of the overall weighted reduction in Gini impurity occurred due to splits utilising WheelType_UNKNOWN. This result aligns well with Task 2.1(E), wherein WheelType_UNKNOWN was utilised by the model as the primary split at the root node.

Attributes with slightly lower levels of importance include; VehOdo (.071), VehBCost (.068), MMRCurrentRetailRatio (0.055) and MMRAcquisitionAuctionAveragePrice (0.047). These results indicate that vehicle type availability, vehicle mileage, purchase price and vehicle value data were significant factors in determining classifications made by the default decision tree.

Each feature importance reflects how much a specific feature helped reduce Gini impurity at every level of the decision tree. Thus, a larger importance value indicates a stronger effect on splitting Good/Bad Buy observations. Importantly, these values do not illustrate if a feature will increase or decrease the odds of a bad buy and neither are they indicative of what portion of correct predictions were generated by a given feature.

Task 2.1(G): Evidence of model overfitting

To evaluate whether the default decision tree demonstrated evidence of overfitting, the decision tree's performance for both training and testing data sets was evaluated and compared to each other as well as to the level of complexity associated with the resultant tree. A model demonstrates "overfitting" when it fit the training data too closely to demonstrate good generalization abilities to new data.

For the default tree, overfitting evidence is present. In addition to achieving perfect training accuracy, the model had an overall test accuracy rate of 81.79%. Therefore, there was a drop-off of about 18.21 percentage points from the training set to the test set. This represents the fact that the tree has been able to closely fit all the training cases; however, the model did not generalise as well to the test set.

Additional evidence supporting the conclusion that the decision tree is demonstrating overfitting behaviour is provided by examining the level of complexity demonstrated by the tree. There were 6,303 total nodes and 3,152 total leaf nodes/rule in the default tree. The parameters used for creating the tree included setting `max_depth=None` which would allow the tree to grow indefinitely until one of two stopping criteria are met. Additionally, the minimum number of samples required per node to split (`min_samples_leaf`) was also set to 1. Finally, `ccp_alpha=0.0` indicated that no cost complexity pruning will be performed. These settings resulted in a very large and complex tree that can fit every unique pattern found in the training data set.

This type of behaviour is indicative of models that have little to no bias but a lot of variance. The model fits the training data very well; however, they tend to be overly sensitive to their respective training sample(s) and thus typically do not perform as well when applied to new data.

Task 2.2 - Decision Tree tuned with GridSearchCV

To reduce the overfitting seen in the default tree model, another Decision Tree was developed utilising GridSearchCV to tune the hyperparameters. GridSearchCV systematically evaluated the different combinations of selected hyper-parameters through cross-validation and determines which combination results in the optimal validation performance. The hyper-parameters examined for tuning are the ones that directly influence the level of complexity of the decision trees.

- **max_depth** was used because there was previously no restriction on how deep the decision tree could grow as specified by `max_depth`.
- **min_samples_split** was varied from its default value of 2, since splitting of a node had occurred previously when there were at least two observations
- **min_samples_leaf** was varied from its default value of 1, since it allows leaves of the default tree to be previously created when they contained a single observation.

The ranges that were selected therefore evaluated a progressive increase in restrictive measures on growing the trees to decrease their model complexity and counteract possible overfitting.

Ten-Fold Cross Validation (`cv = 10`) was used as this gave a reliable estimation of how well the model performs and retained around 90% of all the train data for model fitting during each fold. There was sufficient train data available (29,002 records), and so still retained substantial amounts of both training and test data within each fold.

Task 2.2 (A) - Optimal Decision Tree Parameters

This section explored whether decision trees can be improved (if overfitting observed in the default tree can be reduced) through GridSearchCV by finding optimal combinations of parameters that affect the complexity of trees. `Max_depth`, `min_samples_split` and `min_samples_leaf` were previously controlled due to their influence on the development of tree complexity. The search ranges for these parameters were set to compare them with the default settings.

Figure 2-8: Initial optimal parameters for 'tuned' decision tree

```
Best parameters: {'max_depth': 5, 'min_samples_leaf': 5, 'min_samples_split': 2}
```

The initial GridSearchCV found `max_depth = 5`, `min_samples_leaf = 5`, and `min_samples_split = 2` to be the best combination. Since `max_depth = 5` was the smallest `max_depth` in the initial search space it is possible that a shorter tree would produce better results during cross-validation. As such, a second GridSearchCV was run utilising `max_depth` values from 2-6, and values around the initially determined `min_samples_split` and `min_samples_leaf` values.

Figure 2-9: Final optimal parameters for 'tuned' decision tree

```
Best parameters: {'max_depth': 4, 'min_samples_leaf': 1, 'min_samples_split': 10}
Best cross-validation score: 0.8988692127565999
```

In the second GridSearchCV `max_depth = 4`, `min_samples_leaf = 1`, and `min_samples_split = 10` produced the best combination and resulted in an average cross validation accuracy of approximately 89.89% when splitting into 10 groups. Compared to no restriction on `max_depth` on the default model, `max_depth = 4` provides a significant limitation on tree growth. `min_samples_split = 10` indicates that at least 10 observations must exist within a node prior to additional splits. Lastly, retaining the default `min_samples_leaf = 1` does not limit leaf size. These parameters generated the greatest average cross validation accuracy relative to all parameter combinations in the second GridSearchCV.

Task 2.2 (B) - Classification accuracy of training and test datasets

The performance of the optimised decision tree was calculated again based on the accuracy on both the training and test datasets. This comparison provided insight into how well the optimised model can generalise unseen data.

The trained decision tree was able to achieve a training accuracy of 89.94% and a test accuracy of 89.58%. The test accuracy was only slightly less than the training accuracy; approximately 0.36 percent, showing consistency across both the training data and unseen test data. These results indicate that that model is well-generalised.

In comparison, the default decision tree achieved a training accuracy of 100%, and a test accuracy of 81.79%, thus the tuning resulted in reducing the gap between the training and test accuracy whilst at the same time improving the test accuracy by approximately 7.79% compared to the default decision tree. In this case, the loss in training accuracy is not inherently negative as the default model was shown to fit too tightly to the training data. Therefore, the tuning of the model demonstrated significantly better generalisation to unseen observations.

Task 2.2 (C) - Size of the Chosen Tree

The size and complexity of the optimised decision tree was analysed by identifying the total number of nodes, number of leaf nodes and number of decision rules.

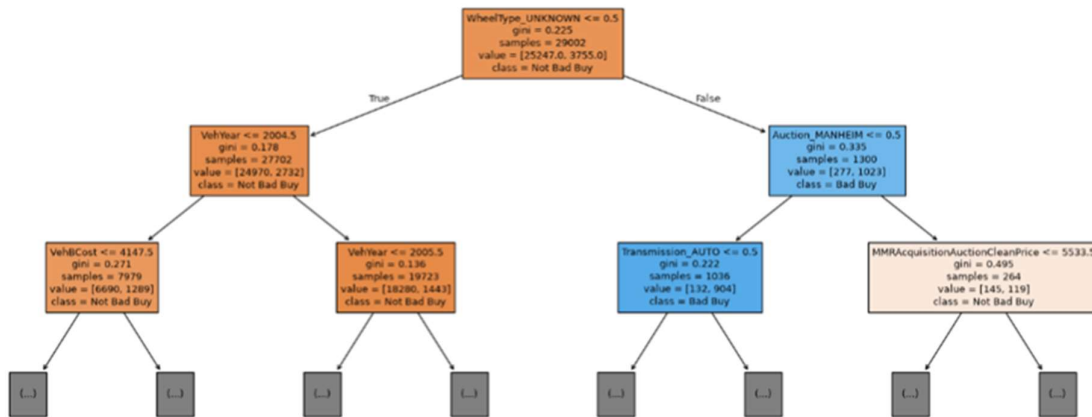
The optimized decision tree has 31 total nodes and 16 terminal leaf nodes, which equates to 16 decision rules. This is a considerable decrease in complexity when comparing to the default decision tree. Our default decision tree had 6,303 total nodes and 3,152 decision rules. These results corresponded well with the hyperparameters identified by GridSearchCV that would best optimize the decision tree, especially those hyperparameters that restricted the maximum depth of the tree (`max_depth = 4`) and required a minimum of 10 observations per node prior to splitting (`min_samples_split = 10`).

The substantially smaller tree is also consistent with the improved generalisation observed in Task 2.2(B), where training and test accuracy were very similar. This suggests that restricting tree complexity reduced the default model's tendency to fit highly specific patterns in the training data.

Task 2.2 (D) - Variables used for first and second split

Following the same process as Task 2.1(E) the first and second splits of the optimised decision tree was generated.

Figure 2-10: View of optimised decision tree at max_depth = 2



WheelType_Unknown was the variable selected for the first split of the tuned decision tree at a threshold of 0.5.

At the second level, the model selected different variables depending on the outcome of the first split. For vehicles with a known wheel type, the second split used VehYear at a threshold of 2004.5. For vehicles with an unknown wheel type, the second split used Auction_MANHEIM at a threshold of 0.5.

These first and second splits are the same as those generated by the default decision tree in Task 2.1(E). This indicates that tuning did not change the variables the model considered most useful for its initial separation of the observations. Instead, the major effect of tuning was likely due to the restriction of subsequent growth of the tree, reducing it from 6,303 nodes and 3,152 rules to 31 nodes and 16 rules.

Task 2.2 (E) - Five important variables in building the tree

Following the same process as Task 2.1(F) the five most important features of the optimised decision tree was produced.

Figure 2-11: Top five important variables for optimised decision tree

```

WheelType_UNKNOWN : 0.8027074936601063
VehYear : 0.0729223037489367
Auction_MANHEIM : 0.05108183435709802
VehBCost : 0.01628745944671066
MMRAcquisitionAuctionAveragePrice : 0.012081588503153088
  
```

The updated results showed that WheelType_UNKNOWN had the greatest influence in determining the splits for the tuned decision tree. It had a score of around .803 which indicated that about 80.3% of the total weighted reduction in Gini Impurity in the optimised

decision tree came from splits based on `WheelType_UNKNOWN`. This matches task 2.1(F), as this was also the feature chosen for the first split.

`VehYear` (.073), `Auction_MANHEIM` (.051), `VehBCost` (.016), and `MMRAcquisitionAuctionAveragePrice` (.012) were the next four features with high enough importance to make the list. Again, both `VehYear` and `Auction_MANHEIM` were also part of the tree structure shown in task 2.2(D) as they made up two of the three second level splits.

Relative to the default decision tree, the optimised tree assigned substantially more weight to `WheelType_UNKNOWN`. `WheelType_UNKNOWN` previously had an importance score of approximately .180 which increased to 0.803 in the optimised tree. Since the optimised tree was pruned by `GridSearchCV` from 6,303 Nodes to 31 Nodes, there were fewer features contributing to the tree's total impurity reduction. As a result, early splits accounted for a much greater proportion of the model's overall feature importance.

Task 2.2 (F) - Evidence of model overfitting

A significant difference between training and test accuracy indicates that the model is overfitting and had fitted the training data more closely than it could generalise to unseen observations.

The optimised decision tree produced a training accuracy of 89.94% whereas producing a test accuracy of 89.58%, which translate to approximately a 0.36% reduction. As such there is very little evidence of overfitting, as the model performed equally well on the training set and the unseen test dataset.

This represents a significantly greater increase than the default decision tree which produced a training accuracy of 100% whilst only achieving a test accuracy of 81.79%. There was a gap of just under 18.21% between these values. In addition, `GridSearchCV` reduced the number of nodes from 6,303 to 31 and the number of rules from 3,152 to 16. Consequently, the tuned version of the model has a substantially smaller number of nodes/rules than the original model. The results suggest that the tuned version of the model demonstrated improved generalisation when applied to previously unseen data. Therefore, the conclusion is that the tuning process has effectively eliminated overfitting evident that was originally present in the default model.

Task 2.3 – Comparing DDT and Optimised tree models

It was established earlier that the optimised decision tree outperformed the default decision tree in terms of its overall performance and generalisation. The default tree achieved 100% training accuracy but only 81.79% test accuracy resulting in a difference of roughly 18.21% and showed signs of overfitting. Conversely, the tuned model produced 89.94% training accuracy and 89.58% test accuracy for a difference of approximately .36%.

Although tuning resulted in a reduction in training accuracy, there was an increase of approximately 7.79 percentage points in test accuracy indicating significantly superior performance when dealing with previously unknown data.

There were substantial differences between the two models regarding their structural complexity. The default tree contained 6,303 nodes and 3,152 rules while the tuned tree contained only 31 nodes and 16 rules. As a result of using `GridSearchCV` to select optimal

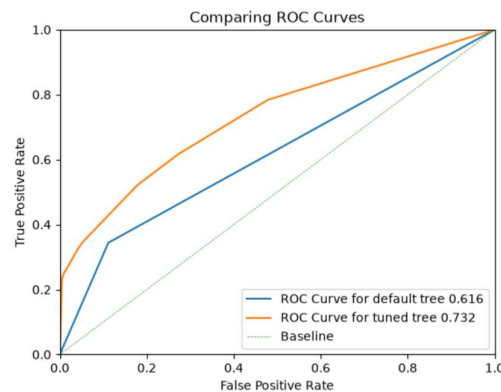
parameters, `max_depth = 4`, `min_samples_split = 10` and `min_samples_leaf = 1` were chosen which greatly restricted the growth of the tree relative to the unrestricted nature of the default model. By limiting the number of levels available to grow into, the amount of overly complex rules that could potentially be generated due to fitting too many unique patterns and/or random noise in the training dataset was dramatically decreased. This resulted in the tuned model having lower variance and exhibiting less overfitting behaviour than the default model which ultimately contributed to its substantially better performance.

The default and optimised decision trees were compared to see what effect hyperparameter tuning had on model complexity, performance generalisation of unseen data. To permit this analysis, ROC-AUC Analysis was used to compare each model's ability to discriminate between `IsBadBuy = 0` and `IsBadBuy = 1` across different thresholds for classification.

Figure 2-11: ROC index for default and tuned models

ROC index on test for default model: 0.6162271386092008
 ROC index on test for tuned model: 0.7321470763825902

Figure 2-12: ROC curve comparison for tuned and default trees



Further evidence supporting the improvement in performance can be seen through the application of Receiver Operator Characteristics (ROC). While the default decision tree yielded an AUC value of 0.616, the optimised decision tree produced an AUC value of 0.732. Moreover, the tuned model's ROC curve tended to lie above the default model's ROC curve across most classification thresholds suggesting a better trade-off between the true positive rate and the false positive rate when varying the classification thresholds. Therefore, based upon the AUC values, it appears that the tuned decision tree can make a better distinction between "Bad Buy" and "Not Bad Buy".

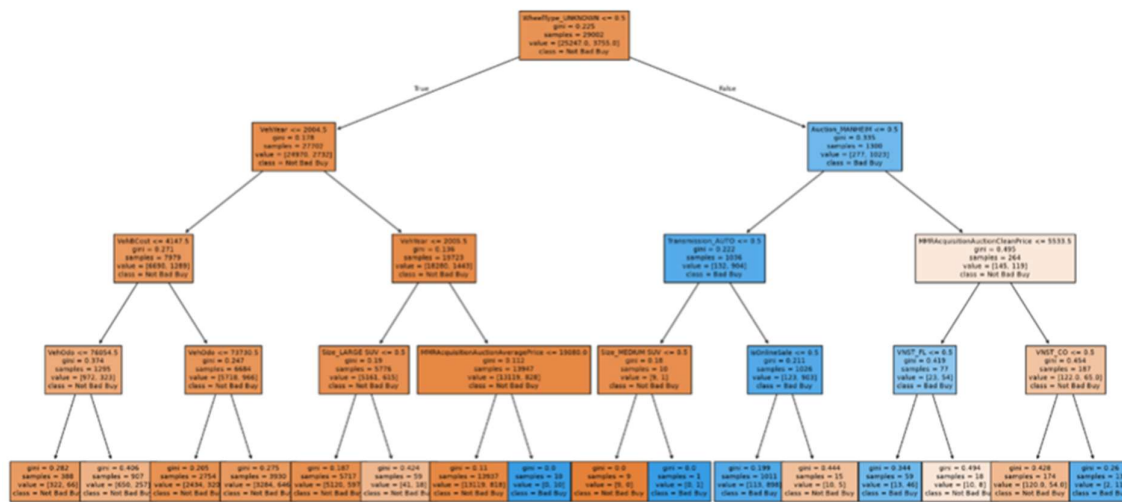
In conclusion, based on performance metrics including, discrimination between target classes, accuracy on unseen test datasets, degree of overfitting exhibited and simplicity/interpretability of resultant trees, the optimised decision tree represents the superior performing model.

Task 2.4 – Cars most likely to be 'kicks'

The optimised decision tree was chosen to find the features that will identify the characteristics of vehicles which are most likely to be identified as "kicks" (`IsBadBuy = 1`). As shown in Task 2.3, the optimised decision tree gave better test accuracy, better ROC-AUC scores and generalisation than the default decision tree. Reviewing the decision path from the root node to each of these leaves provided a set of vehicle characteristics that represented the greatest expected probability of a vehicle being a kick.

To do this, the entire optimised decision tree was plotted. This was possible to graph as the optimised decision tree had its maximum depth restricted to the optimised value of 4.

Figure 2-13: Complete optimised decision tree



The plot of the decision tree shows that `WheelType_UNKNOWN` was found to be the strongest characteristic associated with a vehicle being classified as a "kick". Of the 1,300 observations in the training data with an unknown wheel type, there were 1,023 (78.7%) that were classified as Bad Buys, and while not Bad Buys comprised only 12.9% of all the observations in the training data. The results are consistent with `WheelType_UNKNOWN` also being the feature with the highest importance in terms of classification using the tuned model.

The worst risk combination existed for vehicles with an unknown wheel type, not listed on MANHEIM, automatic transmission, and not sold on-line. In this case, there were 881 out of 899 observations (98.0%) that were classified as Bad Buys.

Other characteristics include:

- **Auction provider** – particularly whether the vehicle was auctioned through MANHEIM.
- **Transmission type** – `Transmission_AUTO` further separated vehicles within the high-risk unknown-wheel-type group. – `Transmission_AUTO` further separated vehicles within the high-risk unknown-wheel-type group.
- **Online sale status** – `IsOnlineSale` further differentiated Bad Buy risk within this group.
- **Vehicle year** – `VehYear` was an important discriminator among vehicles with a known wheel type.
- **Vehicle cost and odometer reading** – `VehBCost` and `VehODO` further separated risk for vehicles with a known wheel type.
- **State** – `VNST_FL` and `VNST_CO` were used to further differentiate vehicles within one auction-provider branch.

In summary, the optimised decision tree shows several characteristics of vehicles that can be interpreted relative to their likelihood of being a "kick". However, these characteristics represent predictive associations rather than causation.

Task 3 – Regression Modelling

Task 3.1 – Processing the dataset for regression modelling

Logistic regression can be used as a viable model because `IsBadBuy` is binary nominal (`Bad Buy = 1` or `Not Bad Buy = 0`). As described in Module 3.11, logistic regression can be applied since it produces predictions using the sigmoid function based upon the probability of an observation being classified into a specific category.

Most preprocessing was completed in Task 1 and 2. To support logistic regression, some variables were required to undergo standardisation to be compared as their numerical values were measured in different units.

The numerical variables that required standardisation were:

- `VehYear`;
- `VehicleAge`;
- `VehOdo`;
- `VehBCost`;
- `MMR Variables`; and
- `OHE Variables`.

Standardisation was fitted using the training data only and then applied to both training and test data to avoid information from the test set influencing model training.

Methodology for Standardisation

Step 1: Display predictor scaling before standardisation

To show the large difference in scales among the predictors, the minimum, maximum, mean, and standard deviation values for the first five predictors in the training dataset were calculated and displayed.

As can be seen in the results table above, there are significant differences in scale among these predictors. For example, `VehYear` had a mean of 2005.36 and a standard deviation of 1.73. On the other hand, `VehOdo` had a mean of 71,324.5 and a standard deviation of 14,759.61. Similarly, the `MMR` value-based predictor variables were also significantly different in terms of scale.

Step 2: Standardise training and test data

The data is normalised by applying a standardiser (`StandardScaler`) to all the predictor variables. First, `StandardScaler` is fitted to the training set (`X_train`) using `fit_transform()`. In doing this, it calculates the mean and std for each predictor, to be able to scale `X_train` appropriately. The same trained scaler is then fitted to `X_test` using `transform()` to obtain the scaled version of `X_test`. In this way, the scaling is based on the training set alone and does therefore not suffer from data leakage.

Step 3: Verify standardisation

Finally, the same statistics from step 1 are again executed and displayed to verify that the standardization has transformed the first 5 predictor variables in `X_train`.

Figure 3-1: Verification of standardisation after scaling

```

After scaling
VehYear - min: -2.52 max: 2.68 mean: 0.0 std dev: 1.0
VehOdo - min: -4.79 max: 27.72 mean: 0.0 std dev: 1.0
MPRAcquisitionAuctionAveragePrice - min: -2.23 max: 11.49 mean: 0.0 std dev: 1.0
MPRAcquisitionAuctionCleanPrice - min: -2.38 max: 11.19 mean: -0.0 std dev: 1.0
MPRAcquisitionRetailAveragePrice - min: -2.31 max: 9.32 mean: 0.0 std dev: 1.0

```

As expected, after applying `StandardScaler`, each of the first five predictor variables now have mean of approximately zero and standard deviations equal to 1, indicating that `StandardScaler` successfully scaled the predictor variables so that they are on similar scales.

Task 3.2 – Regression Modelling with `GridSearchCV`

`GridSearchCV` was used to develop the optimised regression model by identifying which combination of hyperparameters would produce the highest cross-validated performance. Aligning to the module 3 practical, the regularisation parameter `C` was logarithmically tuned. Cross validation value was set at 10.

Task 3.2(A) – Regression Function

The logistic regression function `LogisticRegression()` from Scikit-learn's `linear_model` was used. This type of function is ideal for use when the target variable is binary, so this applies appropriately to our target variable `IsBadBuy`.

Task 3.2(B) – Optimal Parameters

The regularisation parameter `C` was selected for optimisation since it determines the amount of regularisation applied in logistic regression models. Smaller values of `C` result in more regularisation which reduces model complexity, helping to prevent overfitting and improve generalisation to unseen data. Conversely, larger values of `C` apply weaker regularisation. The choice of a range of `C` values (from 0.01 through to 100) allows testing of progressively less strong regularisation across different orders of magnitude. `C`'s permitted ranges can reasonably vary by orders of magnitude. For this reason, testing only a small range of values such as 0.5, 1 and 1.5 would explore only a narrow range of regularisation strengths.

Cross-validation was employed at ten folds to provide an improved estimate of how accurately the model will perform relative to cross-validation at just one-fold. This corresponds to the method of tuning described within the module 3 practical.

The best values for the hyperparameter `C` is shown below:

Figure 3-2: Best values for hyperparameter C

```

Best parameters: {'C': 1}
Best cross-validation score: 0.897110746591544

```

Using `GridSearchCV`, the optimal value for the regularisation parameter was `C = 1`, yielding a best 10-fold cross-validation accuracy of approximately 89.71 %. This implies that the value of `C = 1.0` gave the "best" trade-off between applying regularisation and producing high cross-validation accuracy amongst those values tested (i.e., 0.01, 0.1, 1.0, 10.0, and 100.0).

Task 3.2(C) – Variables Used

Variables that were input into the tuned Logistic Regression model were assessed to confirm the final predictor set that was developed from preprocessing and OHE.

Figure 3-3: Summary of variables used for the Logistic Regression model

```

Number of variables: 120
VehYear
VehOdo
MMRAcquisitionAuctionAveragePrice
MMRAcquisitionAuctionCleanPrice
MMRAcquisitionRetailAveragePrice
MMRAcquisitionRetailCleanPrice
MMRCCurrentAuctionAveragePrice
MMRCCurrentAuctionCleanPrice
MMRCCurrentRetailAveragePrice
MMRCCurrentRetailCleanPrice
MMRCCurrentRetailRatio
VehBCost
IsOnlineSale
WarrantyCost
Auction_ADESA
Auction_MANHEIM

```

The logistic regression model had 120 predictor variables. It contained all the retained numerical predictors (vehicle year, odometer, MMR valuation, vehicle cost; warranty cost) as well as all the categorical predictors that were previously encoded using one-hot encoding.

Task 3.2(D) – Top 5 Important Variables

The five most important variables from the optimized logistic regression model are those whose model coefficients had the largest absolute values. Both positive and negative coefficients may have high absolute values and therefore suggest a strong relationship with the output prediction.

Figure 3-4: Summary of five most important variables

```

MMRAcquisitionAuctionAveragePrice : -1.145093029280855
MMRAcquisitionRetailAveragePrice : 0.8864173175827059
MMRCCurrentRetailCleanPrice : -0.858726967955804
MMRCCurrentAuctionAveragePrice : 0.754924897732235
WheelType_UNKNOWN : 0.7210543613005851

```

MMRAcquisitionAuctionAveragePrice (the price for which vehicles sold at auction) was ranked as the most important variable; this is followed by MMRAcquisitionRetailAveragePrice (vehicles' average retail prices), MMRCCurrentRetailCleanPrice (clean retail price for vehicles), MMRCCurrentAuctionAveragePrice (average price for vehicles at current auctions) and WheelType_UNKNOWN.

Four out of the five most important variables are related to MMR valuations indicating that the vehicle valuation data had an important effect on the logistic regression model. WheelType_UNKNOWN is the one categorical predictor in the top five, and it is also shown to be important by both the default decision tree model and the optimised decision tree model. The sign of each coefficient indicates whether there is a positive or negative association with the log odds of a Bad Buy. We used only the size of each predictor's absolute value when determining its place in our ranking.

Task 3.2(E) – Accuracy of Training and Test Datasets

The classification accuracy of the tuned logistic regression model was evaluated on both the training and test datasets. Comparing these results provides an indication of how well the fitted model performs on the training observations and generalises to unseen test data.

Figure 3-5: Training and Test accuracy of Logistic Regression Model

```

Training accuracy: 0.8974208675263775
Test accuracy: 0.8939662107803701

```

A classification accuracy of approximately 89.74% was obtained for the training data, while an accuracy of 89.40% was obtained for the test data. That the same level of classification accuracy was achieved by the model for both training and test data would indicate that the model maintains its ability to predict accurately with new data. However, there are very slight

differences in accuracy which will be investigated further in Task 3.2(F) where potential overfitting is assessed.

Task 3.2(F) – Overfitting Conclusions

Figure 3-6: Absolute and percentage differences in model accuracy

```
Training accuracy: 0.8974208675263775
Test accuracy: 0.8939662107803701
Accuracy difference: 0.003454656746007423
Accuracy difference (%): 0.3454656746007423
```

The tuned logistic regression model showed very few signs of overfitting. This is supported by the fact that whilst the trained logistic regression model has a training accuracy of 89.74%, the test dataset had an accuracy of 89.40%, which equates to a difference of only approximately 0.35 percentage points. In addition to this small difference in the accuracies, both training and testing accuracy are quite high. Therefore, the overall good performance of the model on the unseen test data set suggests that the model does generalise as well as fits the training data.

Task 3.3 – Minimum variable set Regression Model

Task 3.3(A) – Dimensionality Reduction and Feature Selection

The use of Recursive Feature Elimination with Cross Validation (RFECV) allowed for the reduction in the number of independent variables in the logistic regression model. Recursive elimination is a process where each time one variable is eliminated it produces new models and then those are evaluated using cross validation. This allows for identification of an optimal combination of predictor variables that has high predictive ability.

Figure 3-7: Reduction in variable use using RFECV

```
Original number of variables: 120
Optimal number of variables: 42
```

The application of RFECV resulted in the original 120 variables being reduced to 42 variables by eliminating 78 variables. This type of dimensionality reduction was beneficial since the reduced model performed similarly on the test data as did the full model. The reduced model produced a test accuracy score of 89.43%, which compares to a test accuracy score of 89.40% for the full model. Therefore, the elimination of approximately 65% of the variables retained virtually identical performance to the full model. This resulted in a much smaller set of features than before without reducing the overall performance of the model.

Task 3.3(B) – Variables Included in the Reduced Model

Of the 120 predictors available in the full data set, RFECV chose to retain 42. Of these retained predictors, there was some information about vehicles (such as VehYear, VehOdo, VehBCost and WarrantyCost) along with MMR valuation attributes. In addition, there were one-hot encoded predictor values for auction providers, vehicle makes, wheels, sizes of vehicles, and states. RFECV removed the remainder of the predictors.

Task 3.3(C) – Classification Accuracy

After applying feature reduction, the logistic regression model was retuned using utilizing GridSearchCV with the same logarithmically scaled C parameter search space. This process also utilised the same 10-fold CV method as originally applied. By doing so, both the full (120 features) and reduced (42 features) models were tuned similarly for evaluation.

Figure 3-8: Revaluation of best parameter C after RFECV

Best parameters: {'C': 0.1}
 Best cross-validation score: 0.8972486657395192

Figure 3-9: Updated test and training accuracy values following RFECV

Training accuracy: 0.8973519067650507
 Test accuracy: 0.8942880128720837

The parameters chosen through GridSearchCV resulted in the selection of $C = 0.1$ for the reduced model. The reduced model performed at an approximate level of 89.74% on the training data and at an approximate level of 89.43% on the testing data. These levels of performance are roughly equivalent to those found within the full model which demonstrated similar levels of performance on the training (89.74%) and testing (89.40%) datasets. Therefore, it can be concluded that the application of feature reduction from 120 down to 42 variables had no negative impact upon the overall classification performance.

Task 3.3(D) – Overfitting

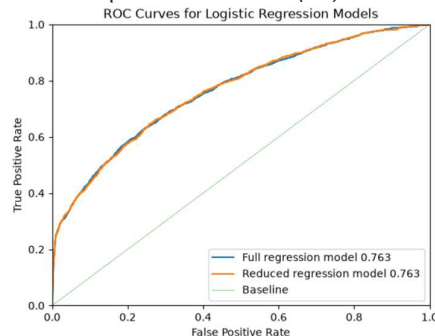
Overfitting potential in the reduced model was evaluated through comparisons of both models' training vs. test (unseen) classification accuracies.

Figure 3-10: Absolute and percentage differences of logistic regression model following RFECV

Accuracy difference: 0.003063893892967018
 Accuracy difference (%): 0.3063893892967018

The results show little indication of overfitting for the reduced model. It had an accuracy of about 89.74% on the training set while it had an accuracy of about 89.43% on the test (unseen data), representing a 0.3% difference. Therefore, this is evidence that the reduced model generalised well from seen data to unseen data. The difference is even less than what was observed by the full model where the difference was close to .35%.

Task 3.4

Figure 3-11: ROC curve comparison for default (full) and RFECV reduced models

Both models achieved almost identically high values on the ROC-AUC score; the full logistic regression model achieved a value of .7630, while the reduced model achieved a slightly lower value of .7629. As they are virtually indistinguishable, these two models have nearly equal discrimination ability for distinguishing between bad buy and not bad buy. Also, because both models stay above the baseline line, we can conclude that they both provide much greater discrimination than would occur randomly.

While there was a very slight difference in the AUC scores (the full model had a slightly higher score), it was minimal. Additionally, although the full model had many more features than the reduced model (120 vs. 42), it had a slightly lower test accuracy (.8940 vs .8943). For this

reason, reduced model is determined to be the better model, since it achieved practically identical performance to that of the full model, but with far fewer features.

The positive coefficients of the reduced logistic regression model were examined to determine which characteristics that were most positively associated with an increased probability that $\text{IsBadBuy} = 1$. It was found that `wheeltype_unknown` (coefficient of 0.758) had the most positively correlated predictor, which means that vehicles that have unknown wheels were most positively associated with being classified as Bad Buys. Some additional positively correlated predictors include some of the MMR valuation measures higher odometer readings.

These characteristics should only be interpreted as associations rather than causes. Specifically, many related MMR variables were included in the model, which makes their individual effects harder to interpret independently. Similarly, an unknown type does not necessarily cause a vehicle to be classified as a bad buy but may instead represent characteristics of that vehicle or lack of information provided in auction listings.

In general, RFECV made significant simplifications to the original logistic regression model without suffering any material losses in predictive performance. Essentially identical ROC curves demonstrate that reducing the feature set from 120 to 42 features resulted in practically no loss of discrimination ability.

Task 4 – Predictive Modelling Using Neural Networks

Task 4.1 – Processing the Dataset for Neural Network Modelling

Most data cleaning such as handling missing values, dropping non-predictive variables, and one-hot encoding categorical features were completed in Tasks 1 to 3. The same training and test splits were retained to maintain a consistent baseline.

Additional preprocessing was required specifically for neural network training.

- **StandardScaler:** Neural networks are sensitive to the differences in the scale of input variables. The scaler was fitted using the function `fit_transform()`
- **Leakage Prevention:** The scaler was fitted strictly on `X_train` using `fit_transform()` and then applied to `X_test` using `transform()`. Fitting across the entire dataset prior to splitting would constitute data leakage by exposing test distribution parameters to the training data.

Task 4.2 – Neural Network Model Using All Predictor Variables

An initial multi-layer perceptron (`MLPClassifier`) was constructed using all 120 predictor variables. 10-fold cross-validation with `GridSearchCV` optimized for ROC-AUC rather than raw accuracy.

Class Imbalance: Roughly 87% of observations belong to the majority class ($\text{IsBadBuy} = 0$). A naive model may predict all zeros with 87% accuracy and be commercially useless. ROC-AUC evaluates ranking discrimination across all classification thresholds regardless of base rates.

Task 4.2(A) – Parameters and Hyperparameter Search

The neural network model was optimised using `GridSearchCV` for optimal combination of the network architecture, activation functions, regularisation strengths, and learning rates. Both one- and two hidden layer networks were searched with varying training parameters to find

how models of differing complexity and learning behaviour could be compared. The base settings used to create NNM `max_iter = 500`; and `random_state = 42`. The following hyperparameters and value ranges were tested in the table below:

Table 4-1: Hyperparameters and values tested for the NNM

HYPERPARAMETER	VALUES TESTED
hidden_layer_sizes	(50,), (100,), (50, 25), (100, 50)
activation	['relu', 'tanh'],
alpha	[0.0001, 0.001, 0.01],
learning_rate_init	[0.001, 0.01]

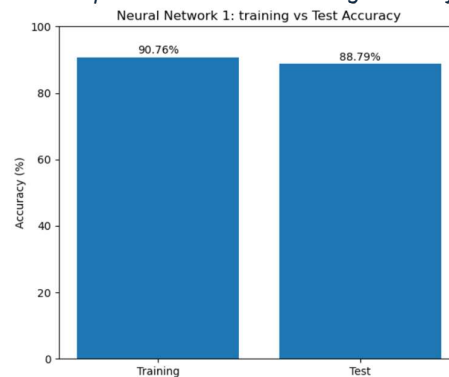
Figure 4-2: Output of best parameters determined by algorithm

Best parameters:
 {'activation': 'relu', 'alpha': 0.01, 'hidden_layer_sizes': (100,), 'learning_rate_init': 0.01}
 Best cross-validation ROC-AUC: 0.7103

Task 4.2(B) – Classification Accuracy

Classification accuracy between the test and training data sets for the NNM were plotted in the figure below.

Figure 4-3: Comparison of test and training accuracy for NNM

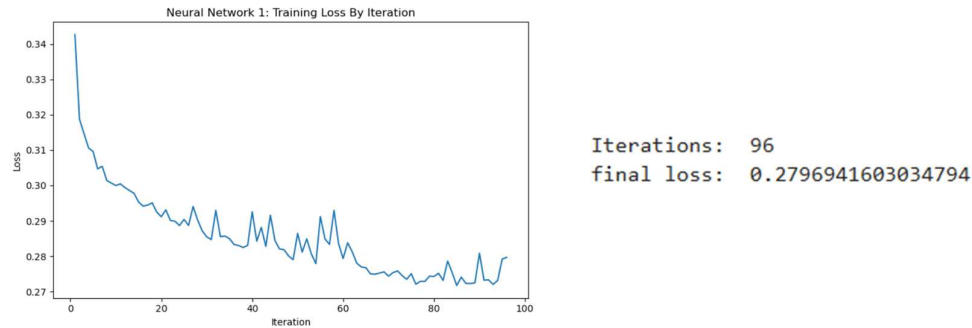


There was a gap of 1.97 % between training and testing datasets. Given the 87% majority in `IsBadBuy = 0`, these figures mask the model's ability to discriminate classes, reinforcing the necessity of ROC-AUC.

Task 4.2(C) – Convergence

Training loss across iterations were assessed for convergence to determine if the Neural Network had reached a point of diminishing returns with regards to Training (i.e., additional training was no longer reducing Loss). The Training Loss Curve remained relatively flat over the course of Iteration, and the optimiser ceased at iteration 96, prior to reaching the maximum of 500 iterations set as a limit, since there was little to no incremental improvement being produced.

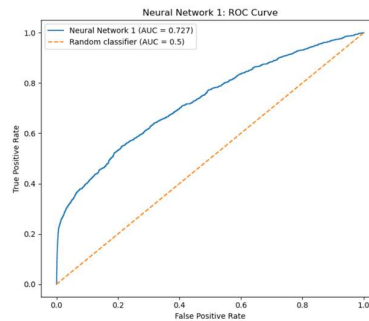
Figure 4-4: Neural Network 1 training loss across iterations, demonstrating convergence after 96 iterations.



Task 4.2(D) – Overfitting

Assessment: Minor overfitting is present.

Figure 4-5: ROC curve for Neural Network 1 on the test dataset



Evidence: The model was able to achieve an accuracy in its training set at 90.76 percent, and an accuracy in the testing set at 88.79 percent, resulting in a 1.97% drop from trained to unseen data. Although this is a modest difference (that can be considered "mild" over-fitting), it does indicate that the model outperforms the test data somewhat more than the training data. However, the model is also performing well enough on the test data, indicating good generalisation.

Task 4.3 – Neural Network Model Using the Reduced Feature Set

A reduction of features, while retaining the total samples, was conducted using the top 5 features identified by Task 2's decision tree feature importance: Features were reduced by 95.8% (from 120 to 5). A separate StandardScaler was fit to the 5-feature training set and transformed onto the test set.

Task 4.3(A) – Feature Selection & Architecture Changes

Feature Selection greatly improved the Model. The reduced number of Inputs (from 120 to 5) significantly reduced the complexity of the Network but enhanced the cross validation roc_auc score from 0.7103 to 0.7517. Therefore, GridSearchCV could choose a smaller Hidden Layer of 50 units instead of 100 units for the original model, along with other choices of Activation, Regularisation, and Learning Rate.

Table 4-6: NN1 vs NN2 Architectural Comparisons

Metric / Setting	NN1 (All 120 Features)	NN2 (Reduced 5 Features)
Input Features	120	5
Hidden Layer Shape	(100,)	(50,)
Activation	relu	tanh

Alpha (L2 Penalty)	0.01	0.001
Initial Learning Rate	0.01	0.001
CV ROC-AUC	0.7103	0.7517

The reduction in Input Features (from 120 to 5), created a much less complex Network Architecture but increased the cross validation roc_auc from .7103 to .7517. This suggests that the top 5 features identified through Decision Tree Tuning had strong predictive properties for the Neural Net Model.

Task 4.3(B) – Classification Accuracy Comparison

NN1 (Full):

Training accuracy: 0.9076

Test accuracy: 0.8879

Gap: 1.97 %

NN2 (Reduced):

Training accuracy: 0.8978

Test accuracy: 0.8958

Gap: 0.20 %

Compared with NN1, NN2 showed a slight decrease in training accuracy (from 90.76% to 89.78%), however a slight increase in Test Accuracy (from 88.79% to 89.58%). Furthermore, the Training-Test Accuracy Gap also decreased from 1.97 to 0.20 percent which indicated that the smaller model generalised better than the larger one to unseen data.

Task 4.3(C) – Number of Training Iterations

NN1

Iterations: 96

Final loss: 0.2796941603034794

NN2:

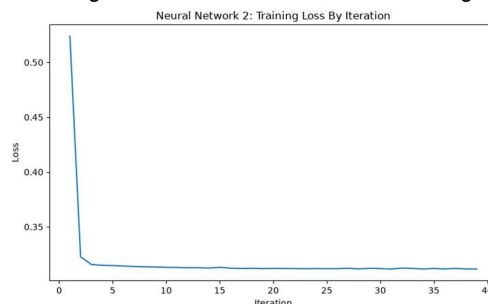
Iterations: 39

Final loss: 0.3114856888441988

Convergence: Full convergence achieved at iteration 39 which is well before the 500 limit.

The NN2 converged at iteration 39, which is 59% less (a 41% decrease) than the 96 iterations needed to reach convergence by the NN1. The Loss Curve was able to drop very quickly and then slowly stabilise, with Training stopping before it had reached its maximum of 500 iterations.

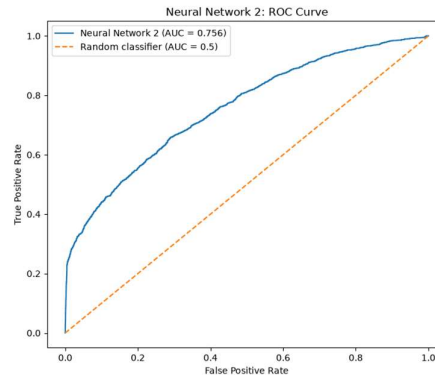
Figure 4-7: Neural Network 2 training loss across iterations, demonstrating convergence after 39 iterations



Task 4.3(D) – Overfitting

There is very little evidence of overfitting. NN2 was able to achieve an almost identical accuracy in both training (89.78%) and testing (89.58%), with a difference of just 0.20 percent. This demonstrates a much smaller gap than NN1's 1.97 percent suggests that the reduced model has demonstrated a far greater ability to generalise to unseen data. It also reached convergence at iteration number 39 which is significantly less than the 500 iterations allowed.

Figure 4-8: ROC curve for Neural Network 2 on the test dataset



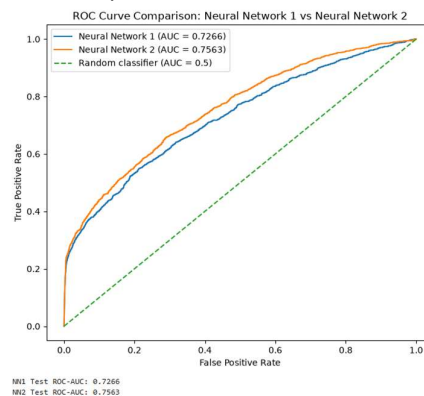
Task 4.4 – ROC Curve Comparison and Characteristics of Kicks

Table 4-9: Performance comparison of the full (NN1) and reduced-feature (NN2) neural network models

Model	Inputs	Train Acc.	Test Acc.	Test ROC-AUC
NN1 (Full)	120	90.76%	88.79%	0.7266
NN2 (Reduced)	5	89.78%	89.58%	0.7563

NN2 outperformed NN1 on the test dataset, achieving higher test accuracy (89.58% vs 88.79%) and a higher ROC-AUC (0.7563 vs 0.7266). It also reduced the training–test accuracy gap from 1.97 to 0.20 percentage points and required fewer training iterations. Therefore, NN2 was selected as the better neural network model.

Figure 4-10: ROC curve comparison of NN1 and NN2, with NN2 performing better



Characteristics of Kicks & Interpretability Limitations

Extracting direct kick characteristics from NN2 illustrates a classic "black-box" trade-off:

- **Non-Linear Entanglement:** Unlike logistic regression (which gives linear log-odds coefficients) or decision trees (which provide explicit if-then split paths), a neural network processes inputs through hidden layer transformations () and dense weight matrices.

- **Feature Interactions:** The impact of VehBCost or VehYear variables on kick probability is volatile because the direction can go either way and are heavily dependent on the simultaneous states of the other inputs.
- **Context of Feature Selection:** The 5 variables selected for use in the best decision tree were WheelType UNKNOWN, Year of Vehicle (VehYear), Auction MANHEIM (Auction_MANHEIM) and Veh Cost (VehBCost) and Acquisition Price at Auction (MMRAcquisitionAuctionAveragePrice). Therefore, these may be considered to be the key input variables into NN2. However, the direction of influence of each variable could not be assessed separately using this method alone. The decision tree from Task 2 provides greater means for interpretability as it produces specific characteristics and their directions.

Overall Task 4 Conclusion

The reduction of 115 variables improved test ROC-AUC from 0.7266 to 0.7563 whilst also reducing convergence time by about 59 %. Neural networks on tabular data can be susceptible to noise therefore excluding uninformative one-hot features allowed NN2 to reach parity with Task 3's reduced logistic regression (0.763).

Task 5 – Stage 1 Final Remarks: Decision-Making

Task 5.1 – Selection of the Best-Performing Model

The predictive models developed in Tasks 2–4 were compared using their performance on the unseen test dataset. The target variable IsBadBuy is imbalanced, with approximately 87% of observations classified as not IsBadBuy, classification accuracy alone was not considered sufficient for selecting the final model. ROC-AUC was also considered because it measures the ability of a model to distinguish between across either of IsBadBuy classifications. The results are summarised below.

Figure 5-1: Accuracy table of all models generated

MODEL	FEATURES	TRAINING ACCURACY	TEST ACCURACY	TRAIN-TEST GAP	TEST ROC-AUC
Default Decision Tree	120	100.00 %	81.79 %	18.21 %	0.616
Optimised Decision Tree	120	89.94 %	89.58%	0.36 %	0.732
Logistic Regression – Full	120	89.74 %	89.40 %	0.34 %	0.763
Logistic Regression – Reduced	42	89.74 %	89.43 %	0.31 %	0.763
Neural Network – Full	120	90.76 %	88.79 %	1.97 %	0.7266
Neural Network – Reduced	5	89.78 %	89.58 %	0.20 %	0.7563

The decision-tree results show clearly why training accuracy should not determine model selection. The default decision tree achieved 100 % training accuracy but only 81.79 % test accuracy and an ROC-AUC of 0.616, which demonstrated substantial overfitting. GridSearchCV reduced the optimised tree to only 31 nodes and 16 rules and improved its test accuracy to 89.58 % and ROC-AUC to 0.732.

The neural network results also demonstrated the benefit of feature reduction. Reducing NN1 from 120 variables to only five variables increased test accuracy from 88.79% to 89.58% and ROC-AUC from 0.7266 to 0.7563.

However, the logistic regression models produced the highest ROC-AUC, at approximately 0.763. The reduced logistic regression achieved the same predictive performance as the full model while reducing the feature set from 120 to 42 variables. Its training accuracy was 89.74 %, and test accuracy was 89.43 %, a difference of only approximately 0.31 %, providing little evidence of overfitting.

ROC Comparison of predictive models

This graph figure above illustrates the comparison of the selected models. The reduced logistic regression achieved in the highest ROC-AUC (approximately 0.763) very narrowly exceeds the reduced neural network (0.7563), while the optimised decision tree achieved 0.732. This indicates that the reduced logistic regression model provided the strongest overall discrimination between a Bad Buy and Not Bad Buy observations, although its advantage over the neural network was small.

Final Model Selection

Based on the combined results, the reduced logistic regression model would be selected for decision-making.

Although the optimised decision tree and reduced neural network both achieved slightly higher test classification accuracy of 89.58 %, compared with 89.43 % for reduced logistic regression, this difference is only 0.15 percentage points. Given the substantial class imbalance in *IsBadBuy*, this small difference in raw accuracy does not provide sufficient reason to prefer these models.

The reduced logistic regression achieved the highest ROC-AUC of approximately 0.763, indicating the strongest overall ability to discriminate between *IsBadBuy* or not. The model also showed very little evidence of overfitting, with only a 0.31 % difference between training and test accuracy. Feature selection using RFECV reduced the original 120 predictors to 42 without materially reducing predictive performance.

Another advantage to logistic regression for decision-making. Its coefficients can be examined to determine both the strength and direction of associations between vehicle characteristics and the probability of a Bad Buy. For example, the reduced model identified *Wheel Type_UNKNOWN* as the strongest positive predictor of *IsBadBuy* risk, together with characteristics including MMR valuation measures, higher odometer readings, medium SUVs and higher warranty costs.

The reduced neural network came close to the logistic regression in predictive performance, achieving an ROC-AUC of 0.7563 using only five variables. However, because of its tendency towards being a “black box”, the relationships of variables are considerably more difficult to interpret. For a model intended to support decision-making, being able to explain why a vehicle has been identified as high risk is valuable.

Therefore, the reduced logistic regression provides the strongest balance between predictive discrimination, generalisation, reduced complexity and interpretability and would be selected as the final decision-making model.

Task 5.2 – Positives and Negatives of the Predictive Modelling Methods

The three modelling approaches demonstrated different strengths and weaknesses during this analysis.

Figure 5-2: Comparison of three modelling approaches

METHOD	POSITIVES	NEGATIVES
Decision Tree	- Highly interpretable - Produces understandable decision rules - Easily identifies interactions between vehicle characteristics - Little preprocessing required - Optimised model generalised well	- Unrestricted trees can overfit - default model produced 100% training but only 81.79% test accuracy - Tuned tree had lower ROC-AUC than logistic regression and reduced neural network.
Logistic Regression	- Highest ROC-AUC - Very little overfitting - Coefficients allow direction and Strength of associations to be examined - Relatively straightforward to explain - RFECV substantially reduced variables without loss of performance	- Assumes relationships can be represented through the logistic model - Preprocessing and standardisation required - Correlated predictors can make individual coefficients difficult to interpret - May not capture complex nonlinear relationships as readily
Neural Network	- Can model complex nonlinear relationships and interactions - Reduced neural network achieved strong ROC-AUC with only five inputs - Feature reduction improved generalisation and reduced training requirements	- Least interpretable model - Requires scaling; more computationally demanding and requires more hyperparameter tuning - Full model showed mild overfitting - Difficult to explain exactly why an individual vehicle receives a particular prediction

Decision Trees

The main advantage of the decision tree was interpretability. The optimised tree allowed specific combinations of vehicle characteristics associated with Bad Buys to be identified directly. For example, `WheelType_UNKNOWN` was the strongest initial predictor and one decision path identified vehicles with unknown wheel type, not auctioned through MANHEIM, automatic transmission and not sold online as a particularly high-risk group.

Its major weakness was susceptibility to overfitting. The unrestricted default tree grew to 6,303 nodes and 3,152 rules, which resulted in perfect training accuracy but substantially poorer performance on unseen observations. Hyperparameter tuning dramatically improved this, demonstrating that tree complexity needed to be carefully controlled.

Logistic Regression

Logistic regression provided the best overall discrimination and good generalisation between training and test data. It also provides more interpretable information than the neural network because coefficients indicate whether variables are positively or negatively associated with `IsBadBuy` probability. More importantly, the reduced model was particularly useful because RFECV removed 78 of the original 120 predictors while maintaining almost identical predictive performance.

A disadvantage is that interpretation becomes more complicated where predictors are correlated. This is particularly relevant to this dataset because several related MMR valuation

variables were retained. Logistic regression is also less naturally suited to modelling complex nonlinear interactions than decision trees or neural networks.

Neural Networks

The neural networks demonstrated their ability to model complex relationships between predictors. The strongest result was obtained after feature selection: NN2 used only five predictors but achieved a test ROC-AUC of 0.7563, compared with 0.7266 for the 120-variable NN1. Its training-test accuracy difference was also reduced from 1.97 to only 0.20 percentage points.

The principal disadvantage is interpretability. Although the five inputs to NN2 are known, the network combines these variables through hidden neurons, weights and nonlinear activation functions. Consequently, it is much harder to explain why a particular vehicle is predicted to be a *IsBadBuy*. This is an important limitation when predictive models are intended to assist human decision-making rather than simply maximise predictive performance.

Overall Conclusion

The analysis demonstrates that the most complex model was not the best model. The default decision tree severely overfitted. The full neural network was outperformed by its five-variable counterpart and reducing the logistic regression from 120 to 42 predictors produced virtually no loss in performance. The reduced logistic regression is therefore the preferred final model. Its ROC-AUC of approximately 0.763 was the highest of the models tested, its test accuracy of 89.43 % was comparable with the highest-performing models and there was little evidence of overfitting. Most importantly for decision-making, it provides a greater capacity to explain which vehicle characteristics are associated with Bad Buy risk than the neural network.

References

- Han, J., Kamber, M., & Pei, J. (2012). *Data Mining : Concepts and Techniques* (3rd ed.). Elsevier/Morgan Kaufmann.
- Devlin, J. (2016, October 12). *28 Jupyter Notebook Tips, Tricks, and Shortcuts for Data Science*. Dataquest. <https://www.dataquest.io/blog/jupyter-notebook-tips-tricks-shortcuts/>